

Network Programming

(Day 3)

Yunmin Go

School of AICE

Handong Global University

Network Programming (Day 3): roadmap

다중화

- I/O Multiplexing Programming
- Various I/O Functions

Alternative of multi-process server

- Disadvantages of multi-process server
 - Frequent process creation causes performance degradation.
 - It is not easy to implement because the flow of multiple processes should be considered.
 - The server implementation becomes more complex if the inter-process communication is required.
- Alternative of multi-process server
 - One process provides services to multiple clients.
 - There must be a way for a single process to handle multiple sockets.
 - IO multiplexing is required

select()

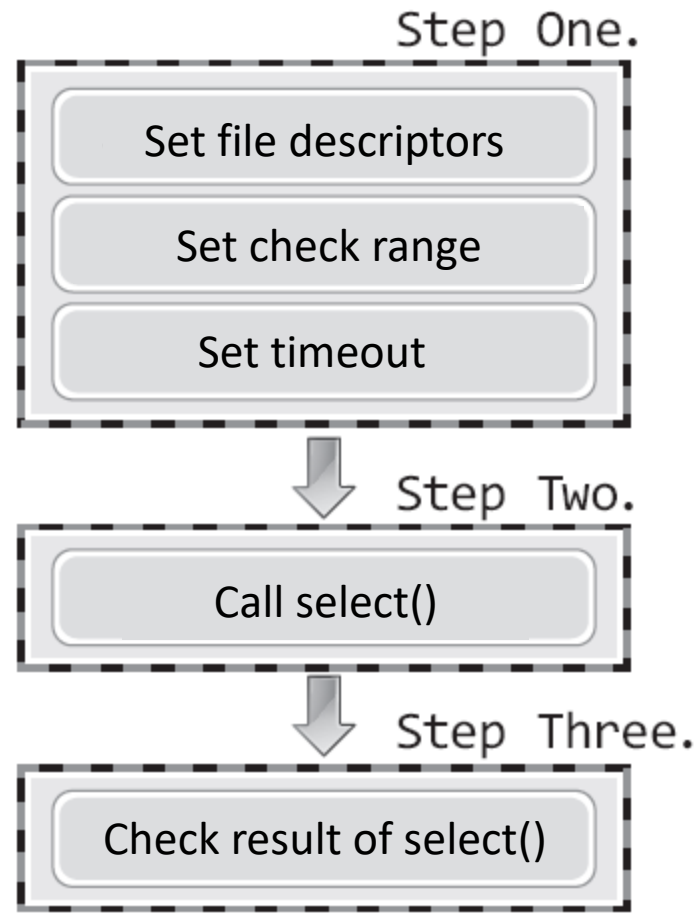
```
#include <sys/select.h>
#include <sys/time.h>
int select(int nfd, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout);
```

```
struct timeval {
    long tv_sec; /* seconds */
    long tv_usec; /* micro-seconds */
}
```

Synchronous I/O multiplexing

- **Waits** for any one of multiple **events** (file descriptors) to occur during a specified amount of time
- **nfd**: the highest-numbered file descriptor in any of the three sets, plus 1
 - select() can monitor only file descriptors numbers that are less than FD_SETSIZE.
- **readfds (writefds)**: The file descriptors in this set are watched to see if they are ready for reading (writing).
- **exceptfds**: The file descriptors in this set are watched for “exceptional conditions”.
- **timeout**: interval that select() should block waiting for a file descriptor to become ready.
 - If timeout is Null, wait forever
 - If timeout is 0, do not wait at all.
- **Return value**
 - Success: the number of descriptors in the set on success
 - 0 if the **timeout** was reached
 - Error: -1

Call procedures of select()



Readable? Writable? Exception?

Descriptor sets

- fd_set: array of integers
 - Each bit in each integer corresponds to a descriptor
 - The related macro functions

Macro function	Description
void FD_ZERO(fd_set *set)	Clear all entities from the set
void FD_SET(int fd, fd_set *set)	Add fd to the set
void FD_CLR(int fd, fd_set *set)	Remove fd to the set
int FD_ISSET(int fd, fd_set *set)	Return true if fd is in the set

```
int main(void)
{
```

```
    fd_set set;
```

```
    FD_ZERO(&set);
```

fd0	fd1	fd2	fd3	...
0	0	0	0	

```
    FD_SET(1, &set);
```

fd0	fd1	fd2	fd3	...
0	1	0	0	

```
    FD_SET(2, &set);
```

fd0	fd1	fd2	fd3	...
0	1	1	0	

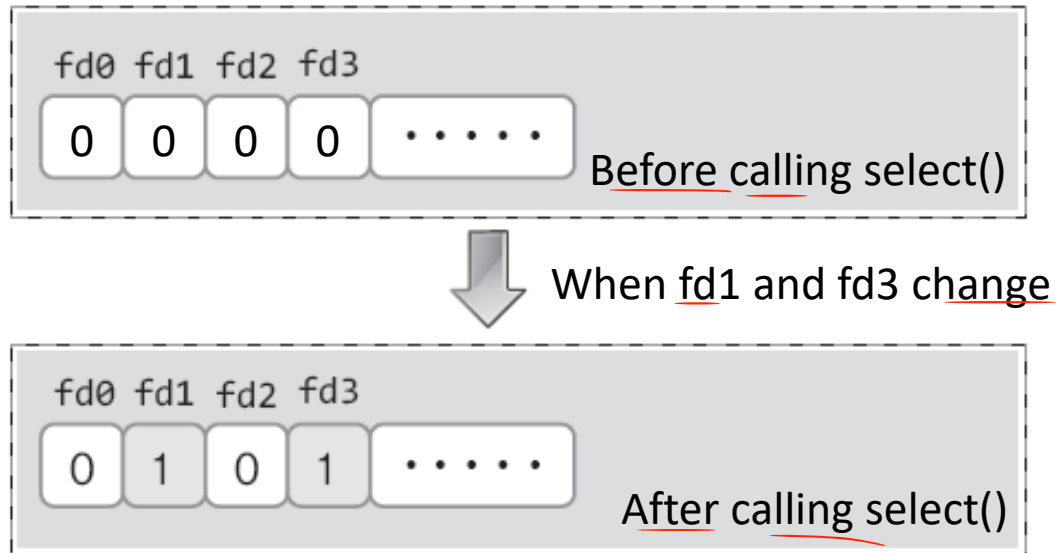
```
    FD_CLR(2, &set);
```

fd0	fd1	fd2	fd3	...
0	1	0	0	

```
}
```

Detecting changes of file descriptor

- How to know which file descriptors are ready?



Detecting changes of file descriptor

- Conditions that cause a socket to be ready for select

Condition	Readable?	Writable?	Exception?
Data to read	O		
New connection ready from listening socket	O		
Read-half closed (FIN received)	O		
Space available for writing		O	
Write-half closed		O	
TCP out-of-band data			O

Example: select.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/time.h>
#include <sys/select.h>

#define BUF_SIZE 30

int main(int argc, char *argv[])
{
    fd_set reads, temps;
    int result, str_len;
    char buf[BUF_SIZE];
    struct timeval timeout;

    FD_ZERO(&reads);
    FD_SET(0, &reads); // 0 is standard input(console)
```

```
while(1)
{
    temps = reads;
    timeout.tv_sec = 5;
    timeout.tv_usec = 0;

    result = select(1, &temps, 0, 0, &timeout);
    if (result == -1)
    {
        puts("select() error!");
        break;
    }
    else if (result == 0)
    {
        puts("Time-out!");
    }
    else
    {
        if (FD_ISSET(0, &temps))
        {
            str_len = read(0, buf, BUF_SIZE);
            buf[str_len] = 0;
            printf("message from console: %s", buf);
        }
    }
}
return 0;
}
```

sd관점에서 어떤 신호인가? 에 따라서 다르게 read

Example: echo_selectserv.c

```
int main(int argc, char *argv[])
{
    int serv_sock, clnt_sock;
    struct sockaddr_in serv_adr, clnt_adr;
    struct timeval timeout;
    fd_set reads, cpy_reads;

    socklen_t adr_sz;
    int fd_max, str_len, fd_num, i;
    char buf[BUF_SIZE];
    .....

    FD_ZERO(&reads);
    FD_SET(serv_sock, &reads);
    fd_max = serv_sock;

    while (1)
    {
        cpy_reads = reads;
        timeout.tv_sec = 5;
        timeout.tv_usec = 5000;

        if ((fd_num = select(fd_max+1, &cpy_reads, 0, 0, &timeout)) == -1)
            break;

        if (fd_num == 0)
            continue;
```

```
        for (i = 0; i < fd_max+1; i++)
        {
            if (FD_ISSET(i, &cpy_reads))
            {
                if (i == serv_sock) // connection request!
                {
                    adr_sz = sizeof(clnt_adr);
                    clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_adr, &adr_sz);
                    FD_SET(clnt_sock, &reads);
                    if (fd_max < clnt_sock)
                        fd_max = clnt_sock;
                    printf("connected client: %d \n", clnt_sock);
                }
                else // read message!
                {
                    str_len = read(i, buf, BUF_SIZE);
                    if (str_len == 0) // close request!
                    {
                        FD_CLR(i, &reads);
                        close(i);
                        printf("closed client: %d \n", i);
                    }
                    else
                    {
                        write(i, buf, str_len); // echo!
                    }
                }
            }
        }
    }
}
```

Example: file_server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define BUF_SIZE 30
void error_handling(char *message);

int main(int argc, char *argv[])
{
    int serv_sd, clnt_sd;
    FILE * fp;
    char buf[BUF_SIZE];
    int read_cnt;

    struct sockaddr_in serv_adr, clnt_adr;
    socklen_t clnt_adr_sz;

    if (argc != 2) {
        printf("Usage: %s <port>\n", argv[0]);
        exit(1);
    }

    fp = fopen("file_server.c", "rb");
    serv_sd = socket(PF_INET, SOCK_STREAM, 0);
```

```
    memset(&serv_adr, 0, sizeof(serv_adr));
    serv_adr.sin_family = AF_INET;
    serv_adr.sin_addr.s_addr = htonl(INADDR_ANY);
    serv_adr.sin_port = htons(atoi(argv[1]));

    bind(serv_sd, (struct sockaddr*)&serv_adr, sizeof(serv_adr));
    listen(serv_sd, 5);

    clnt_adr_sz = sizeof(clnt_adr);
    clnt_sd = accept(serv_sd, (struct sockaddr*)&clnt_adr, &clnt_adr_sz);

    while(1)
    {
        read_cnt = fread((void*)buf, 1, BUF_SIZE, fp);
        if (read_cnt < BUF_SIZE)
        {
            write(clnt_sd, buf, read_cnt);
            break;
        }
        write(clnt_sd, buf, BUF_SIZE);
    }

    shutdown(clnt_sd, SHUT_WR);
    read(clnt_sd, buf, BUF_SIZE);
    printf("Message from client: %s\n", buf);

    fclose(fp);
    close(clnt_sd); close(serv_sd);
    return 0;
}
```

Example: file_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define BUF_SIZE 30
void error_handling(char *message);

int main(int argc, char *argv[])
{
    int sd;
    FILE *fp;

    char buf[BUF_SIZE];
    int read_cnt;
    struct sockaddr_in serv_adr;
    if (argc != 3) {
        printf("Usage: %s <IP> <port>\n", argv[0]);
        exit(1);
    }
```

```
    fp = fopen("receive.dat", "wb");
    sd = socket(PF_INET, SOCK_STREAM, 0);

    memset(&serv_adr, 0, sizeof(serv_adr));
    serv_adr.sin_family = AF_INET;
    serv_adr.sin_addr.s_addr = inet_addr(argv[1]);
    serv_adr.sin_port = htons(atoi(argv[2]));

    connect(sd, (struct sockaddr*)&serv_adr, sizeof(serv_adr));

    while ((read_cnt = read(sd, buf, BUF_SIZE)) != 0)
        fwrite((void*)buf, 1, read_cnt, fp);

    puts("Received file data");
    write(sd, "Thank you", 10);
    fclose(fp);
    close(sd);
    return 0;
}
```

실습#1

- select()를 활용한 File 전송 서버 구현
 - Day1의 file_server.c를 select() 기반의 Multiple access server로 구현해보세요.
 - Client는 프로그램은 Day1의 file_client.c를 그대로 사용해도 됩니다.
 - 서버가 동시에 여러 클라이언트를 서비스 할 수 있는지 검증해야합니다.

Pros and cons of select()

■ Advantage of select()

- Good compatibility
 - Various operating systems support select().
 - Changing select() from a Linux server to a Windows server is easy
- Easy to use

■ Disadvantage of select()

- We should use loop statements to check all file descriptors we want to check.
- Whenever the select() is called, a parameter must be passed to observe the file descriptor.

Solution of select()

- epoll

- No need of loop statements to check changes of file descriptors.
- No need of passing parameters to observe the file descriptors.
- Kernel maintains status of file descriptors.
- Linux only

- There are three functions to implement epoll

- epoll_create
- epoll_ctl
- epoll_wait

epoll_create()

```
#include <sys/epoll.h>
int epoll_create(int size);
```

Open an epoll file descriptor

- Creates a new epoll instance
- *size*: since Linux 2.6.8, the size is ignored, but must be greater than zero
 - Nowadays, the kernel dynamically sizes the required data structures without needing the hint.
- Return value
 - Success: file descriptor (> 0)
 - Error: -1

epoll_ctl()

```
#include <sys/epoll.h>
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);
```

Control interface for an epoll file descriptor

- Used to add, modify, or remove entries in the interest list of the epoll instance
- *epfd*: interest list of the epoll instance
- *op*: operation to be performed
 - EPOLL_CTL_ADD: add an entry to the interest list of *epfd*. The entry includes *fd*
 - EPOLL_CTL_MOD: change *fd* in the interest list to the new setting specified in *event*
 - EPOLL_CTL_DEL: Remove *fd* from the interest list
- *fd*: target file descriptor
- *event*: describes the object linked to the file descriptor *fd*
- Return value
 - Success: 0
 - Error: -1
- Example
 - `epoll_ctl(A, EPOLL_CTL_ADD, B, C);`
 - `epoll_ctl(A, EPOLL_CTL_DEL, B, NULL);`

struct epoll_event

- `epoll_event` is used to register event types and detect events

```
struct epoll_event {
    uint32_t  events;  // epoll events
    epoll_data_t data; // User data variable
};

typedef union epoll_data {
    void      *ptr;
    int        fd;
    uint32_t   u32;
    uint64_t   u64;
} epoll_data_t;
```

- *events*: a bit mask composed by ORing together zero or more of the available event types
- *data*: specifies data that the kernel should save and then return (via `epoll_wait()`) when this file descriptor becomes ready.
- Example

```
struct epoll_event event;
...
event.events = EPOLLIN;
event.data.fd = sockfd;
epoll_ctl(epfd, EPOLL_CTL_ADD, sockfd, &event);
...
```

struct epoll_event

■ Event types

Types	Description
EPOLLIN	The associated <u>file</u> is <u>available</u> for <u>read</u> operations
EPOLLOUT	The associated <u>file</u> is <u>available</u> for <u>write</u> operations
EPOLLPRI	Out-of-band data on a TCP socket (See POLLPRI)
EPOLLRDHUP	Stream socket peer closed connection, or shut down writing half of connection
EPOLLERR	Error condition happened on the associated file descriptor. This event is also reported for the write end of a pipe when the read end has been closed.
EPOLLET	Requests edge-triggered notification for the associated file descriptor.
EPOLLONESHOT	Requests one-shot notification for the associated file descriptor. This means that after an event notified for the file descriptor by <code>epoll_wait()</code> , the file descriptor is disabled in the interest list and no other events will be reported by the <code>epoll</code> interface. The user must call <code>epoll_ctl()</code> with <code>EPOLL_CTL_MOD</code> to rearm the file descriptor with a new event mask.

epoll_wait()

```
#include <sys/epoll.h>
```

```
int epoll_wait(int epfd, struct epoll_event *events, int maxevents, int timeout);
```

Wait for an I/O event on an epoll file descriptor

- Waits for events on the epoll instance
- *epfd*: interest list of the epoll instance
- *events*: the buffer pointed to by *events* is used to return information from the ready list about file descriptor in the interest list that have some events available
- *maxevents*: up to *maxevents* are returned by `epoll_wait()`. It must be greater than zero.
- *timeout*: number of milliseconds that `epoll_wait()` will block. If *timeout* is -1, `epoll_wait()` will block until event occurs.
- Return value
 - Success: the number of file descriptors ready for the requested I/O
 - Error: -1

Example: echo_epollserv.c

```
int main(int argc, char *argv[])
{
    ...
    struct epoll_event *ep_events;
    struct epoll_event event;
    int epfd, event_cnt;
    ...
    epfd=epoll_create(EPoll_SIZE);
    ep_events=malloc(sizeof(struct epoll_event)*EPoll_SIZE);

    event.events=EPOLLIN;
    event.data.fd=serv_sock;
    epoll_ctl(epfd, EPOLL_CTL_ADD, serv_sock, &event);

    while(1)
    {
        event_cnt=epoll_wait(epfd, ep_events, EPoll_SIZE, -1);
        if(event_cnt==-1)
        {
            puts("epoll_wait() error");
            break;
        }
    }
}
```

```
for(i=0; i<event_cnt; i++)
{
    if(ep_events[i].data.fd==serv_sock)
    {
        adr_sz=sizeof(clnt_adr);
        clnt_sock=accept(serv_sock, (struct sockaddr*)&clnt_adr, &adr_sz);
        event.events=EPOLLIN;
        event.data.fd=clnt_sock;
        epoll_ctl(epfd, EPOLL_CTL_ADD, clnt_sock, &event);
        printf("connected client: %d \n", clnt_sock);
    }
    else
    {
        str_len=read(ep_events[i].data.fd, buf, BUF_SIZE);
        if(str_len==0) // close request!
        {
            epoll_ctl(epfd, EPOLL_CTL_DEL, ep_events[i].data.fd, NULL);
            close(ep_events[i].data.fd);
            printf("closed client: %d \n", ep_events[i].data.fd);
        }
        else
        {
            write(ep_events[i].data.fd, buf, str_len); // echo!
        }
    }
}
...
}
```

Triggering modes

- epoll provides both edge-triggered and level-triggered modes.
- The level-triggered mode generates events continuously while data remains in the input buffer regardless of the amount of data in the input buffer.
- The edge-triggered method generates an event only once when data enters the input buffer.



(a) Response to positive level



(b) Positive-edge response



(c) Negative-edge response

Example: echo_EPLTserv.c

event count 만큼만 반복문으로 돌
(!= not 전체)

```
#define BUF_SIZE 4
#define EPOLL_SIZE 50
...

int main(int argc, char *argv[])
{
    ...
    epfd=epoll_create(EPOLL_SIZE);
    ep_events=malloc(sizeof(struct epoll_event)*EPOLL_SIZE);

    event.events=EPOLLIN;
    event.data.fd=serv_sock;
    epoll_ctl(epfd, EPOLL_CTL_ADD, serv_sock, &event);

    while(1)
    {
        event_cnt=epoll_wait(epfd, ep_events, EPOLL_SIZE, -1);
        if(event_cnt== -1)
        {
            puts("epoll_wait() error");
            break;
        }

        puts("return epoll_wait");
    }
}
```

```
for(i=0; i<event_cnt; i++)
{
    if(ep_events[i].data.fd==serv_sock)
    {
        adr_sz=sizeof(clnt_adr);
        clnt_sock=accept(serv_sock, (struct sockaddr*)&clnt_adr, &adr_sz);
        event.events=EPOLLIN;
        event.data.fd=clnt_sock;
        epoll_ctl(epfd, EPOLL_CTL_ADD, clnt_sock, &event);
        printf("connected client: %d \n", clnt_sock);
    }
    else
    {
        str_len=read(ep_events[i].data.fd, buf, BUF_SIZE);
        if(str_len==0) // close request!
        {
            epoll_ctl(epfd, EPOLL_CTL_DEL, ep_events[i].data.fd, NULL);
            close(ep_events[i].data.fd);
            printf("closed client: %d \n", ep_events[i].data.fd);
        }
        else
        {
            write(ep_events[i].data.fd, buf, str_len); // echo!
        }
    }
}
...
}
```

For edge-triggered mode based server

- Change to non-blocking I/O

```
int flag = fcntl(fd, F_GETFL, 0);  
fcntl(fd, F_SETFL, flag|O_NONBLOCK);
```

- Obtain socket options by using fcntl() and set socket options with O_NONBLOCK
- In edge-triggered mode, event occurs only once when data is received. Thus, when an event occurs, all data must be read by a buffer with sufficient size. In other words, delay due to I/O may occur depending on the amount of data. So, we use non-blocking I/O for edge-trigger to perform other operations concurrently.

For edge-triggered mode based server

■ Check input buffer

```
#include <error.h>  
int errno;
```

- When the buffer reads data based on the non-blocking I/O, we must check whether data receiving is completed.
- We must include error.h and check errno.
- If errno has EAGAIN, the buffer is empty.

Example: echo_EPETserv.c

```
#define BUF_SIZE 4
#define EPOLL_SIZE 50
...

int main(int argc, char *argv[])
{
    ...
    epfd=epoll_create(EPOLL_SIZE);
    ep_events=malloc(sizeof(struct epoll_event)*EPOLL_SIZE);

    setnonblockingmode(serv_sock);
    event.events=EPOLLIN;
    event.data.fd=serv_sock;
    epoll_ctl(epfd, EPOLL_CTL_ADD, serv_sock, &event);

    while(1)
    {
        event_cnt=epoll_wait(epfd, ep_events, EPOLL_SIZE, -1);
        if(event_cnt==-1)
        {
            puts("epoll_wait() error");
            break;
        }

        puts("return epoll_wait");
        for(i=0; i<event_cnt; i++)
        {
            if(ep_events[i].data.fd==serv_sock)
            {
                adr_sz=sizeof(clnt_adr);
```

```
void setnonblockingmode(int fd)
{
    int flag=fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, flag|O_NONBLOCK);
}
```

```
adr_sz=sizeof(clnt_adr);
clnt_sock=accept(serv_sock, (struct sockaddr*)&clnt_adr, &adr_sz);
setnonblockingmode(clnt_sock);
event.events=EPOLLIN|EPOLLET;
event.data.fd=clnt_sock;
epoll_ctl(epfd, EPOLL_CTL_ADD, clnt_sock, &event);
printf("connected client: %d \n", clnt_sock);
}
else
{
    while(1)
    {
        str_len=read(ep_events[i].data.fd, buf, BUF_SIZE);
        if(str_len==0) // close request!
        {
            epoll_ctl(epfd, EPOLL_CTL_DEL, ep_events[i].data.fd, NULL);
            close(ep_events[i].data.fd);
            printf("closed client: %d \n", ep_events[i].data.fd);
            break;
        }
        else if(str_len<0)
        {
            if(errno==EAGAIN)
                break;
        }
        else
        {
            write(ep_events[i].data.fd, buf, str_len); // echo!
        }
    }
}
```

실습#2

- Epoll을 활용한 File 전송 서버 구현
 - 실습#1을 epoll을 활용하여 재구현하세요.

Network Programming (Day 3): roadmap

- I/O Multiplexing Programming
- Various I/O Functions

recv() & send()

```
#include <sys/types.h>
#include <sys/socket.h>
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
```

■ Flags

Options	Description	recv	Send
MSG_DONTROUTE	Don't use a gateway to send out the packet, send to hosts only on directly connected networks.		0
MSG_DONTWAIT	Enables nonblocking operation	0	0
MSG_OOB	Send or receive out-of-band data (Urgent)	0	0
MSG_PEEK	Return data from the beginning of the receive queue without removing that data from the queue	0	
MSG_WAITALL	Wait for all the data (Block until the full request is satisfied)	0	

Example: oob_send.c & oob_recv.c

oob_recv.c

```
int main(int argc, char *argv[])
{
    ...
    act.sa_handler = urg_handler;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;
    ...
    serv_adr_sz = sizeof(serv_adr);
    recv_sock = accept(acpt_sock, (struct sockaddr*)&serv_adr, &serv_adr_sz);

    fcntl(recv_sock, F_SETOWN, getpid());
    state = sigaction(SIGURG, &act, 0);

    while ((str_len = recv(recv_sock, buf, sizeof(buf), 0)) != 0)
    {
        if (str_len == -1)
            continue;
        buf[str_len] = 0;
        puts(buf);
    }
    ...
}
```

```
void urg_handler(int signo)
{
    int str_len;
    char buf[BUF_SIZE];
    str_len = recv(recv_sock, buf, sizeof(buf)-1, MSG_OOB);
    buf[str_len] = 0;
    printf("Urgent message: %s \n", buf);
}
```

oob_send.c

```
int main(int argc, char *argv[])
{
    ...
    write(sock, "123", strlen("123"));
    send(sock, "4", strlen("4"), MSG_OOB);
    write(sock, "567", strlen("567"));
    send(sock, "890", strlen("890"), MSG_OOB);
    ...
}
```

Result

```
[gym@localhost practice09]$ ./oob_recv 9190
Urgent message: 4
Urgent message: 0
123
56789
```

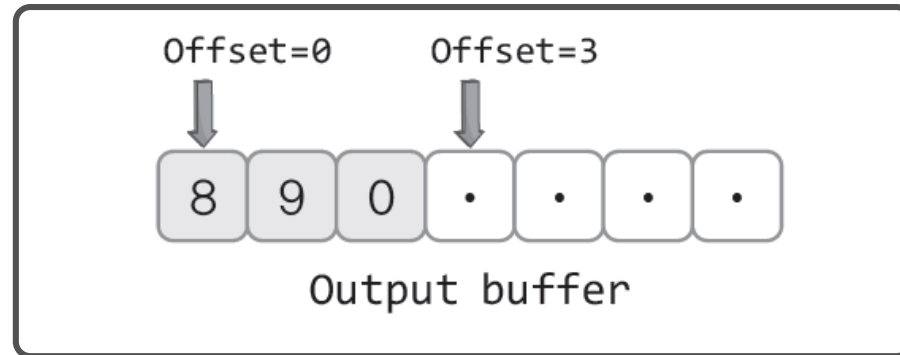
```
[gym@localhost practice09]$ ./oob_send 10.1.0.1 9190
```

Data for OOB (TCP Urgent mode)

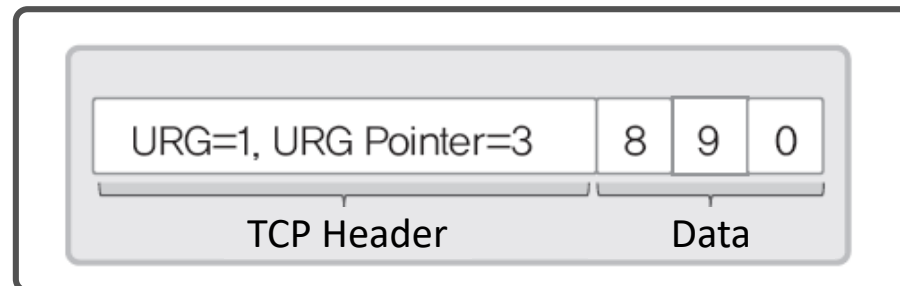
```
send(sock, "890", strlen("890"), MSG_OOB);
```



Output buffer



TCP Packet



If the message with URG_OOB is delivered, the operating system generates the SIGURG signal to inform that **urgent** processing of the message is necessary.

URG=1: indicates that there is urgent message in this packet
URG Pointer=3: location information of urgent message

Example: peek_send.c & peek_recv.c

peek_recv.c

```
int main(int argc, char *argv[])
{
    ...
    recv_sock = accept(acpt_sock, (struct sockaddr*)&recv_adr, &recv_adr_sz);

    while (1)
    {
        str_len = recv(recv_sock, buf, sizeof(buf)-1, MSG_PEEK|MSG_DONTWAIT);
        if (str_len > 0)
            break;
    }

    buf[str_len] = 0;
    printf("Buffering %d bytes: %s \n", str_len, buf);

    str_len = recv(recv_sock, buf, sizeof(buf)-1, 0);
    buf[str_len] = 0;
    printf("Read again: %s \n", buf);
    ...
}
```

oob_send.c

```
int main(int argc, char *argv[])
{
    ...
    if (connect(sock, (struct sockaddr*)&send_adr, sizeof(send_adr)) == -1)
        error_handling("connect() error!");

    write(sock, "123", strlen("123"));
    ...
}
```

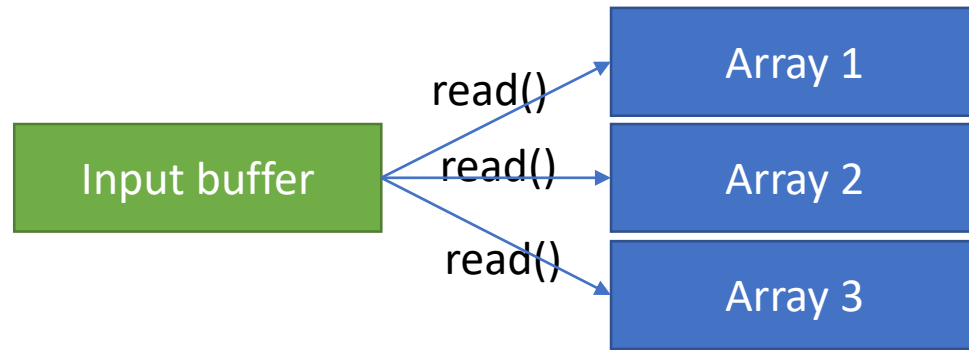
Result

```
[gym@localhost practice09]$ ./peek_recv 9190
Buffering 3 bytes: 123
Read again: 123
```

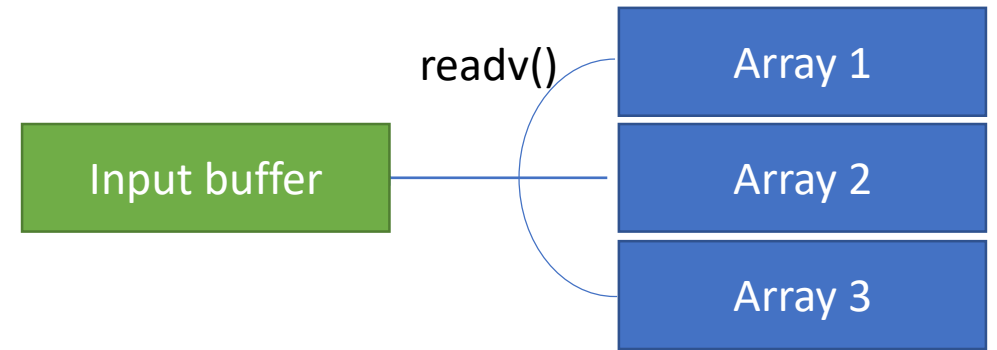
```
[gym@localhost practice09]$ ./peek_send 10.1.0.1 9190
```


readv() & writev()

■ readv()

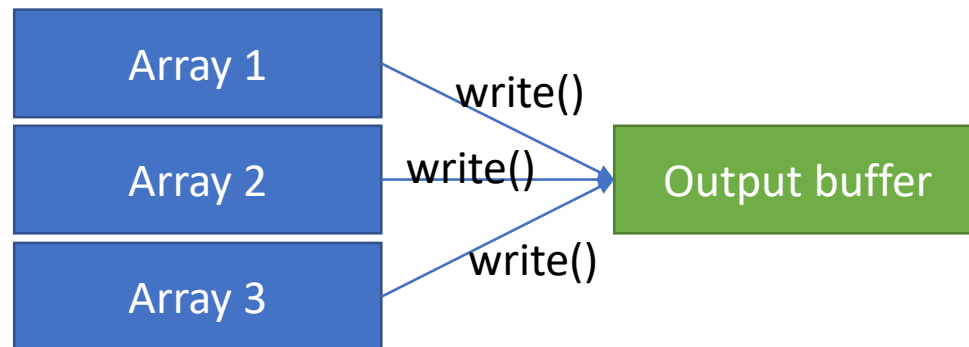


< read function >

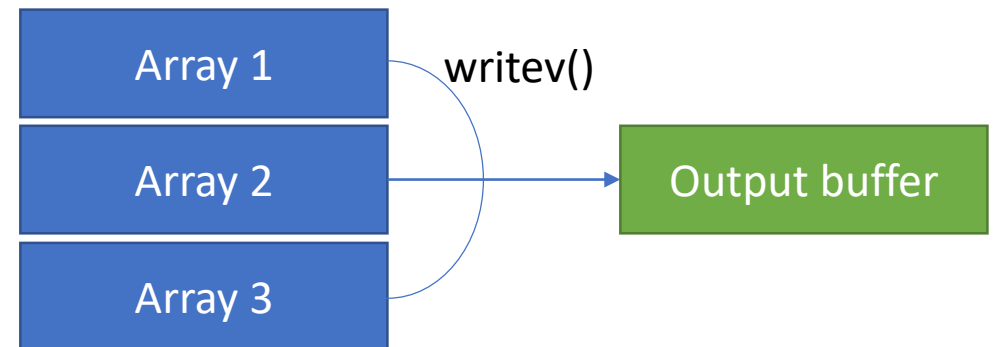


< readv function >

■ writev()



< write function >



< writev function >

readv() & writev()

```
#include <sys/uio.h>
ssize_t readv(int fd, const struct iovec *iov, int iovcnt);
ssize_t writev(int fd, const struct iovec *iov, int iovcnt);
```

Read or write data into multiple buffer

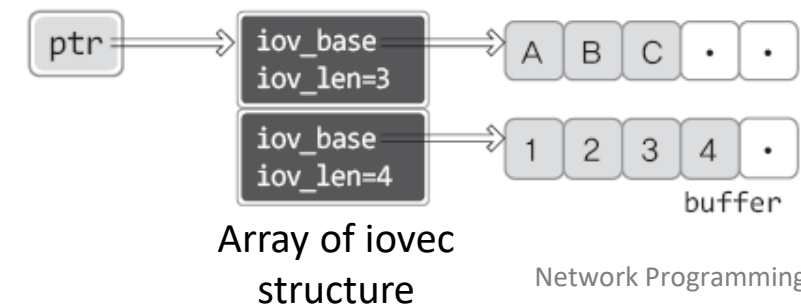
- readv and writev let us read into or write from one or more buffers with a single function call
- *fd*: file descriptor
- *iov*: address of iovec structure

```
struct iovec {
    void *iov_base; /* Starting address */
    size_t iov_len; /* Number of bytes to transfer */
};
```
- *iovcnt*: number of arrays

- Return value

- Success:
 - readv: the number of bytes read
 - writev: the number of byte written
- Error: -1

```
writev( 1 , ptr , 2 );
```



Example: readv.c & writev.c

readv.c

```
#include <stdio.h>
#include <sys/uio.h>
#define BUF_SIZE 100

int main(int argc, char *argv[])
{
    struct iovec vec[2];
    char buf1[BUF_SIZE] = {0,};
    char buf2[BUF_SIZE] = {0,};
    int str_len;

    vec[0].iov_base = buf1;
    vec[0].iov_len = 5;
    vec[1].iov_base = buf2;
    vec[1].iov_len = BUF_SIZE;

    str_len = readv(0, vec, 2);
    printf("Read bytes: %d \n", str_len);
    printf("First message: %s \n", buf1);
    printf("Second message: %s \n", buf2);
    return 0;
}
```

writev.c

```
#include <stdio.h>
#include <sys/uio.h>

int main(int argc, char *argv[])
{
    struct iovec vec[2];
    char buf1[] = "ABCDEFGH";
    char buf2[] = "1234567";
    int str_len;

    vec[0].iov_base = buf1;
    vec[0].iov_len = 3;
    vec[1].iov_base = buf2;
    vec[1].iov_len = 4;

    str_len = writev(1, vec, 2);
    puts("");
    printf("Write bytes: %d \n", str_len);
    return 0;
}
```

Result: readv.c

```
[gym@localhost practice09]$ ./readv
Handong global university!
Read bytes: 27
First message: Hando
Second message: ng global university!
```

Result: writev.c

```
[gym@localhost practice09]$ ./writev
ABC1234
Write bytes: 7
```

Socket and standard I/O

- The standard I/O stream can be used with sockets, but there are a few items to consider.
 - A standard I/O stream can be created from any descriptor by calling the `fdopen` function.
 - Similarly, given a standard I/O stream, we can obtain the corresponding descriptor by calling `fileno` function.
 - `fseek`, `fsetpos`, `rewind` functions is that they all call `lseek`, which fails on a socket.
 - The easiest way to handle the read-write problem is to open two standard I/O stream for a given socket: one for reading and one for writing.

Standard I/O

- Standard I/O functions
 - Use a file pointer (FILE *)
 - Example: fopen(), fclose(), fwrite(), fread()
- System functions
 - Use a file descriptor (int)
 - Example: open(), close(), write(), read()

fdopen()

sd -> file pointer 변환

```
#include <stdio.h>
FILE *fdopen(int fildes, const char *mode);
```

Associate a stream with a file descriptor

- associate a stream with a file descriptor.
- *fildes*: file descriptor
- *mode*:
 - r: open a file for reading
 - w: open file for writing
 - a: open file for writing at end-of-file
 - r+: open a file for reading and writing
 - w+: open a file for reading and writing
 - a+: open a file for reading and writing at end-of-file
- Return value
 - Success: pointer to a stream
 - Error: null pointer

fileno()

```
#include <stdio.h>
int fileno(FILE *stream);
```

file 포인터 -> sd 소켓 디스크립터

Map a stream pointer to a file descriptor

- return the integer file **descriptor** associated with the **stream** pointed to by stream.
- *stream*: file descriptor
- Return value
 - Success: integer value of the file **descriptor** associated with **stream**
 - Error: null pointer

Example: todes.c

```
#include <stdio.h>
#include <fcntl.h>

int main(void)
{
    FILE *fp;
    int fd = open("data.dat", O_WRONLY|O_CREAT|O_TRUNC);
    if (fd == -1)
    {
        fputs("file open error", stdout);
        return -1;
    }

    printf("First file descriptor: %d \n", fd);
    fp = fdopen(fd, "w");
    fputs("TCP/IP SOCKET PROGRAMMING \n", fp);
    printf("Second file descriptor: %d \n", fileno(fp));
    fclose(fp);
    return 0;
}
```

Result

```
[gym@localhost practice09]$ ./todes
First file descriptor: 3
Second file descriptor: 3
[gym@localhost practice09]$ cat data.dat
TCP/IP SOCKET PROGRAMMING
```


Example: echo_stdserv.c & echo_stdclnt.c

echo_stdserv.c

```
int main(int argc, char *argv[])
{
    ...
    for (i = 0; i < 5; i++)
    {
        clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_adr, &clnt_adr_sz);
        if (clnt_sock == -1)
            error_handling("accept() error");
        else
            printf("Connected client %d \n", i+1);

        readfp = fdopen(clnt_sock, "r");
        writefp = fdopen(clnt_sock, "w");

        while (!feof(readfp))
        {
            fgets(message, BUF_SIZE, readfp);
            fputs(message, writefp);
            fflush(writefp);
        }
        fclose(readfp);
        fclose(writefp);
    }
    ...
}
```

echo_stdclnt.c

```
int main(int argc, char *argv[])
{
    ...
    readfp = fdopen(sock, "r");
    writefp = fdopen(sock, "w");

    while (1)
    {
        fputs("Input message(Q to quit): ", stdout);
        fgets(message, BUF_SIZE, stdin);
        if (!strcmp(message, "q\n") || !strcmp(message, "Q\n"))
            break;

        fputs(message, writefp);
        fflush(writefp);
        fgets(message, BUF_SIZE, readfp);
        printf("Message from server: %s", message);
    }
    fclose(writefp);
    fclose(readfp);
    ...
}
```

Separate Input and Output Streams

- It is common to open two standard I/O stream for a given socket: one for reading and one for writing.
 - For the easy handling
 - However, we need to carefully handles the duplicated file descriptors when close the connection.

Problem when closing

- Even though we get **FILE pointers** of read and write for one socket, if we call the **fclose()** on one of them, the socket will close completely, **not half-close**.

단순히 두 포인터 생성해 사용

-> 하나라도 fclose() 시, 참조하는 원본 소켓, 파괴 (half-close 불가)

sep_serv.c

```
readfp = fdopen(clnt_sock, "r");
writefp = fdopen(clnt_sock, "w");

fputs("FROM SERVER: Hi~ client? \n", writefp);
fputs("I love all of the world \n", writefp);
fputs("You are awesome! \n", writefp);
fflush(writefp);

fclose(writefp);
fgets(buf, sizeof(buf), readfp);
```

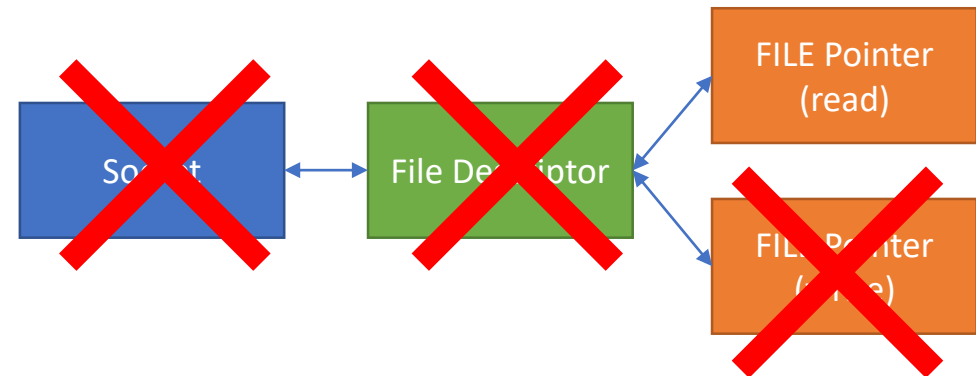
fgets() will not be executed.

sep_clnt.c

```
readfp = fdopen(sock, "r");
writefp = fdopen(sock, "w");

while (1)
{
    if (fgets(buf, sizeof(buf), readfp)==NULL)
        break;
    fputs(buf, stdout);
    fflush(stdout);
}

fputs("FROM CLIENT: Thank you! \n", writefp);
fflush(writefp);
fclose(writefp); fclose(readfp);
```



```
[gym@localhost practice09]$ ./sep_serv 9190
```

```
[gym@localhost practice09]$ ./sep_clnt 127.0.0.1 9190
FROM SERVER: Hi~ client?
I love all of the world
You are awesome!
```

dup() & dup2()

```
#include <unistd.h>
int dup(int oldfd);
int dup2(int oldfd, int newfd);
```

Duplicate a file descriptor

- The dup() system call creates a copy of the file descriptor *oldfd*, using the lowest-numbered unused file descriptor for the new descriptor.
- The dup2() system call performs the same task as dup(), but instead of using the lowest-numbered unused file descriptor, it uses the file descriptor number specified in *newfd*.
- Return value
 - Success: new file descriptor
 - Error: -1

Solution when closing

- **Half-close** can be achieved by duplicating file descriptor.

sep_serv2.c

```
readfp = fdopen(clnt_sock, "r");
writefp = fdopen(dup(clnt_sock), "w");

fputs("FROM SERVER: Hi~ client? \n", writefp);
fputs("I love all of the world \n", writefp);
fputs("You are awesome! \n", writefp);
fflush(writefp);

shutdown(fileno(writefp), SHUT_WR);
fclose(writefp);

fgets(buf, sizeof(buf), readfp); fputs(buf, stdout);
fclose(readfp);
```

fgets() will be executed.

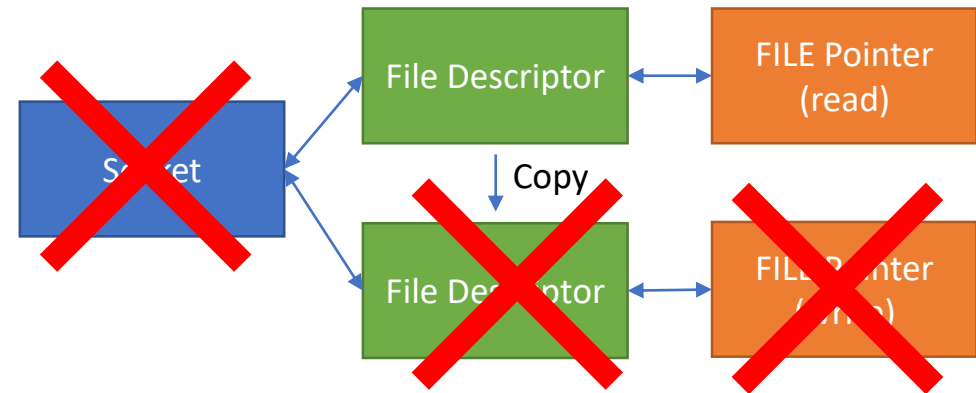
sep_clnt.c

```
readfp = fdopen(sock, "r");
writefp = fdopen(sock, "w");

while (1)
{
    if (fgets(buf, sizeof(buf), readfp)==NULL)
        break;
    fputs(buf, stdout);
    fflush(stdout);
}

fputs("FROM CLIENT: Thank you! \n", writefp);
fflush(writefp);
fclose(writefp); fclose(readfp);
```

```
[gym@localhost practice09]$ ./sep_serv2 9190
FROM CLIENT: Thank you!
```



The socket can be closed only when all related file descriptors are closed.

```
[gym@localhost practice09]$ ./sep_clnt 127.0.0.1 9190
FROM SERVER: Hi~ client?
I love all of the world
You are awesome!
```

Questions #3-1

1. `select()` 함수에서 `fd_set`을 초기화하고 재사용할 때 주의해야 할 점은 무엇이며, 루프 내에서 매번 `FD_ZERO()`와 `FD_SET()`을 호출하는 이유는 무엇인가?
2. `select()` 함수의 `nfds` 인자는 무엇을 의미하며, 이 값의 선택이 서버의 성능과 자원 관리에 어떤 영향을 미치는가?
3. `select()` 기반 서버에서 새로 접속한 클라이언트의 소켓을 적절히 `fd_set`에 추가하고 관리하기 위한 자료구조 설계 방식을 설명하라.
4. `epoll` 시스템 콜이 `select()`보다 성능상 유리한 이유는 무엇이며, 특히 대규모 클라이언트 처리 상황에서 어떤 차이를 보이는가?

Questions #3-2

5. `epoll_create()`, `epoll_ctl()`, `epoll_wait()` 함수의 역할을 각각 설명하고, 이들 함수 호출 순서를 기반으로 `epoll` 서버의 동작 흐름을 기술하라.
6. `epoll`의 ET(Edge Triggered) 방식과 LT(Level Triggered) 방식의 차이점을 비교하고, ET 방식 사용 시 주의해야 할 구현상의 특징을 서술하라.
7. `epoll_event` 구조체의 `data` 필드를 활용하여 연결된 소켓의 상태나 클라이언트 정보를 추적하는 방법을 설명하라.
8. `select()`와 `epoll`의 리눅스 커널 내부 처리 구조의 차이는 무엇이며, 각각의 방식이 갖는 구조적 장단점을 설명하라.
9. `epoll`을 사용할 때 `epoll_ctl()`의 `EPOLL_CTL_MOD`를 사용하는 대표적인 예제 상황은 무엇이며, 해당 변경이 필요한 이유는 무엇인가?

Questions #3-3

10. 실시간성 및 스케일 확장 측면에서 select() 기반 서버가 갖는 구조적 한계는 무엇이며, epoll 방식이 이를 어떻게 보완하는가?
11. epoll_wait()의 타임아웃 값을 -1, 0, 양수로 설정했을 때 각각의 의미와 사용 시나리오를 설명하라.
12. select_server.c와 epoll_server.c의 구조를 비교하고, 각 코드가 클라이언트 접속과 데이터 수신을 처리하는 방식의 차이점을 설명하라.
13. epoll 기반 서버에서 클라이언트로부터 대량의 데이터를 처리하는 경우, read()를 반복 호출하는 루프를 어떤 방식으로 구성해야 효율적인가? thread 생성-> 다른 cpu core 에서 처리하는 방식 (기존의 fd 입력받을 thread / data 처리)
14. epoll을 이용한 서버 구현에서 EPOLL_CTL_DEL을 소홀히 했을 때 발생할 수 있는 문제점을 실제 동작 예를 들어 설명하라.

숙제 #3

■ Simple Remote Shell 프로그램 구현

- 클라이언트가 서버에 접속하여 서버의 디렉토리와 파일 정보를 확인할 수 있는 프로그램을 작성하세요.
- 클라이언트가 서버에 접속하면, 서버는 서버 프로그램이 실행되고 있는 디렉토리 위치 및 파일 정보(파일명 + 파일 크기)를 클라이언트에게 전달합니다.
- 클라이언트는 서버로부터 수신한 정보들을 출력합니다.
- 클라이언트는 서버의 디렉토리를 변경해가며 디렉토리 정보와 디렉토리 안에 있는 파일 정보(파일명 + 파일크기)를 확인할 수 있습니다.
- 클라이언트가 원하는 파일을 서버로부터 다운로드 받을 수 있어야 합니다.
- 클라이언트는 자신이 소유한 파일을 서버에게 업로드 할 수 있어야 합니다.
- 클라이언트가 서버에 접속하면 클라이언트는 서버 프로그램의 권한과 동일한 권한을 갖습니다. 즉, 서버 프로그램에게 허용되지 않은 디렉토리와 파일은 클라이언트도 접근할 수 없습니다.
- 서버는 여러 클라이언트의 요청을 동시에 지원할 수 있어야 합니다.
→ I/O Multiplexing 사용!
- Makefile을 만들어서 컴파일 할 수 있어야 합니다.